RKNN2_Toolkit_Lite_Python_API_参考

文件标识：RK-KF-YF-439

发布版本：V2.5.0

日期：2026-09-30

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园 A 区 18 号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文是 RKNN Toolkit Lite2 Python API 参考手册。

RKNN Toolkit Lite2 为 Rockchip NPU 平台提供 Python 编程接口，主要用于在设备端加载和运行 RKNN 模型。RKNN 模型需要先通过 RKNN Toolkit2 完成模型转换。

本手册以 `RKNNLite` 公开接口为主体，介绍 RKNN Toolkit Lite2 的适用环境、基本调用流程、模型加载、Runtime 初始化、模型推理、SDK 版本查询、模型支持平台查询以及资源释放等内容。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V1.2.0	Raul	2022-01-13	初始版本。	Vincent
V1.3.0	Raul	2022-04-27	更新版本号。	Vincent
V1.4.0	Raul	2022-08-30	更新 `init_runtime` 接口 `core_mask` 参数的说明。	Vincent
V1.5.0	Raul	2023-05-18	1. 更新适配芯片，新增 RK3562； 2. 更新适配的 Python 版本，增加 ARM64 平台 Python 3.8 和 Python 3.10 的支持。	Vincent
V1.5.2	Raul	2023-08-21	更新版本号。	Vincent
V1.6.0	Raul	2023-10-10	1. 新增适配 AARCH64 Python 3.11 的安装包； 2. 推理接口增加对动态 Shape 模型的支持； 3. 推理接口增加对 4 维 NCHW 格式输入的支持。	Vincent
V1.6.2	Raul	2024-03-14	1. 新增对 RK3576 的支持； 2. 调整 `init_runtime` 接口 `core_mask` 属性的说明。	Vincent
V2.5.0	HPC	2026-09-30	1. 文档结构重构； 2. 新增加密模型加载相关说明； 3. 新增 `reuse_outputs` 参数说明。	Vincent

目录

1. API 参考说明
    1.1 适用版本和平台
    1.2 安装与环境部署
    1.3 基本使用流程
    1.4 示例程序
    1.5 API 返回约定
2. RKNNLite 编程模型
    2.1 对象生命周期
    2.2 模型、Runtime 和推理的关系
    2.3 输入数据、数据布局和输出
        2.3.1 inputs
        2.3.2 data_type 和 data_format
        2.3.3 inputs_pass_through
        2.3.4 get_frame_id
        2.3.5 reuse_outputs
        2.3.6 推理输出与失败返回
    2.4 NPU Core 配置
    2.5 查询接口和资源释放
3. RKNNLite 类
    3.1 对象初始化和释放
        init
        release
    3.2 模型加载与 Runtime 初始化
        load_rknn
        init_runtime
    3.3 模型推理
        inference
    3.4 设备、版本和平台查询
        list_devices
        get_sdk_version
        list_support_target_platform
4. 异常与错误处理
    4.1 模型加载和 Runtime 初始化错误
    4.2 推理输入与返回值检查

4.3 NPU Core 配置检查

4.4 日志输出

5. 附录

5.1 参考文档

5.2 问题反馈渠道

1. API 参考说明

本手册用于说明 RKNN Toolkit Lite2 的 Python API。内容以 `RKNNLite` 对象为主体，重点整理环境要求、模型加载、Runtime 初始化、模型推理、SDK 版本查询和模型支持平台查询等接口。

RKNN Toolkit Lite2 主要用于 RKNN 模型在 Rockchip NPU 上的部署。模型转换由 RKNN Toolkit2 完成，本手册不展开模型转换流程。

1.1 适用版本和平台

本文档对应 RKNN Toolkit Lite2 V2.5.0。

当前支持范围如下：

项目	支持范围
RKNN Toolkit Lite2 版本	V2.5.0
硬件平台	RK3562、RK3566系列、RK3568系列、RK3576系列、RK3588系列
操作系统	Debian 10 / 11 / 12（aarch64）
Python	3.8 / 3.9 / 3.10 / 3.11 / 3.12
Python 库依赖	`numpy`、`ruamel.yaml`、`psutil`

1.2 安装与环境部署

RKNN Toolkit Lite2 支持通过 `pip3 install` 安装。

如果系统中没有安装 `python3`、`pip3` 等程序，可先安装基础依赖：

```
sudo apt-get update
sudo apt-get install -y python3 python3-dev python3-pip gcc
```

部分依赖模块安装时需要编译源码，因此同时安装 `python3-dev` 和 `gcc`，可避免后续依赖编译失败。

示例中还会使用 `opencv-python` 和 `numpy`：

```
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
```

注：

1. RKNN Toolkit Lite2 本身并不依赖 `opencv-python`，但示例需要使用该模块处理图像。
2. 在 Debian 10 固件上直接通过 `pip3` 安装 `numpy` 可能失败，建议使用上述系统包安装方式。

RKNN Toolkit Lite2 各平台安装包位于 SDK 的 `packages` 文件夹中。进入该目录后，根据 Python 版本安装对应 Wheel：

```
# Python 3.8
pip3 install rknn_toolkit_lite2-2.x.0-cp38-cp38-linux_aarch64.whl

# Python 3.9
pip3 install rknn_toolkit_lite2-2.x.0-cp39-cp39-linux_aarch64.whl

# Python 3.10
pip3 install rknn_toolkit_lite2-2.x.0-cp310-cp310-linux_aarch64.whl

# Python 3.11
pip3 install rknn_toolkit_lite2-2.x.0-cp311-cp311-linux_aarch64.whl

# Python 3.12
pip3 install rknn_toolkit_lite2-2.x.0-cp312-cp312-linux_aarch64.whl
```

1.3 基本使用流程

在使用 RKNN Toolkit Lite2 之前，需要先通过 RKNN Toolkit2 将各深度学习框架导出的模型转换为 RKNN 模型。

RKNN Toolkit Lite2 的基本调用流程如下：

```
创建 RKNNLite 对象
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
准备输入并完成预处理
    ↓
inference()
    ↓
处理推理结果
    ↓
release()
```

其中：

- `RKNNLite()`：创建 RKNN Toolkit Lite2 Python 对象；
- `load_rknn()`：加载 RKNN 模型；
- `init_runtime()`：初始化 Runtime 运行环境；
- `inference()`：对输入数据执行模型推理；
- `release()`：使用结束后释放 RKNNLite 对象所占用的资源。

调用 `inference()` 前，需要准备输入数据并完成相应预处理，再根据实际输入信息设置 `data_format` 等参数。推理完成后，通常还需要对输出结果进行后处理，以完成上层应用功能。

1.4 示例程序

SDK 的 `examples` 目录提供 `resnet18` 示例。该示例加载 resnet18 模型并进行推理，输出测试图片的 Top-5 分类结果。

运行步骤如下：

1. 准备一块已经安装 RKNN Toolkit Lite2 的开发板；
2. 将示例推送到开发板；
3. 进入 `examples/resnet18` 目录并运行：

```
python3 test.py
```

参考运行结果：

```
-----TOP 5-----
[812]: 0.999442994594574
[404]: 0.0004096269840374589
[657]: 3.284541890025139e-05
[833]: 2.6112385967280716e-05
[895]: 1.8509887013351545e-05
```

1.5 API 返回约定

根据 `RKNNLite` 当前 Python 实现，各主要 API 的返回形式如下：

接口	正常返回	失败或特殊返回
`load_rknn()`	`0`	`-1`
`list_devices()`	`(adb_devices, ntb_devices)`，两项均为 `list`	未单独定义失败返回值
`init_runtime()`	`0`	参数、模型加载、Runtime 创建或初始化失败时返回 `-1`
`inference()`	`list`，成员为输出 `ndarray`；`get_frame_id=True` 时会在返回列表末尾追加 Frame ID	Runtime 未初始化或设置输入、运行、获取输出失败时返回 `None`
`get_sdk_version()`	SDK 版本信息字符串	Runtime 未初始化或查询失败时返回 `None`
`list_support_target_platform()`	`dict`，由平台查询函数返回	指定的 RKNN 模型路径不存在时返回 `None`
`release()`	`None`	`None`

对于返回状态码的接口，应显式检查返回值。例如：

```
ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_AUTO)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

对于 `inference()`、`get_sdk_version()` 和 `list_support_target_platform()`，建议同时检查是否返回 `None`，避免在前置条件不满足或内部执行失败时继续处理无效结果。

2. RKNNLite 编程模型

`RKNNLite` 是 RKNN Toolkit Lite2 的 Python 主入口。应用通过该对象完成 RKNN 模型加载、Runtime 初始化、模型推理、版本信息查询以及资源释放。

RKNN Toolkit Lite2 面向设备侧模型部署。使用前，需要先通过 RKNN Toolkit2 将各深度学习框架导出的模型转换为 RKNN 模型；设备侧应用再按照 `RKNNLite()` → `load_rknn()` → `init_runtime()` → `inference()` → `release()` 的顺序完成模型运行。

理解模型、Runtime、输入数据和 NPU Core 配置之间的关系，有助于正确使用第 3 章中的接口。

2.1 对象生命周期

RKNN Toolkit Lite2 的基本使用流程如下：

```
创建 RKNNLite 对象
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
准备输入数据并完成预处理
    ↓
inference()
    ↓
处理推理结果
    ↓
release()
```

其中：

- `RKNNLite()`：创建 RKNNLite 对象，可通过 `verbose` 和 `verbose_file` 配置详细日志；
- `load_rknn()`：加载指定路径的 RKNN 模型；
- `init_runtime()`：初始化模型运行所需的 Runtime 环境；
- `inference()`：对准备好的输入数据执行推理，并返回输出结果；
- `release()`：RKNNLite 对象使用完成后释放相关资源。

`init_runtime()` 会检查模型是否已通过 `load_rknn()` 加载；`inference()` 会检查 Runtime 是否已初始化。因此，正常调用顺序应为 `load_rknn()` → `init_runtime()` → `inference()`。

推荐按照以下方式组织应用代码：

```
rknn_lite = RKNNLite(verbose=True)

ret = rknn_lite.load_rknn('./model.rknn')
if ret != 0:
    print('Load RKNN model failed')
    exit(ret)

ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)

outputs = rknn_lite.inference(inputs=[input_data])

# 根据模型类型处理 outputs
# ...

rknn_lite.release()
```

对于输入数据，调用 `inference()` 前需要完成相应预处理，并根据输入维度和实际数据排列方式设置 `data_format`。推理完成后，通常还需要对输出结果进行后处理，以完成上层应用功能。

2.2 模型、Runtime 和推理的关系

在 RKNN Toolkit Lite2 中，可以将模型部署过程理解为以下层次：

- **模型转换阶段**：使用 **RKNN Toolkit2** 将深度学习框架模型转换为 **RKNN 模型**；
- **模型加载阶段**：通过 `load_rknn()` 将 **RKNN 模型** 加载到 `RKNNLite` 对象；
- **运行时初始化阶段**：通过 `init_runtime()` 初始化 **Runtime 环境**；
- **推理执行阶段**：通过 `inference()` 获取 **推理结果**。

RKNN Toolkit2 和 RKNN Toolkit Lite2 分别承担不同阶段的工作：

阶段	工具 / 接口	作用
模型转换	RKNN Toolkit2	将深度学习框架导出的模型转换为 RKNN 模型
模型加载	`load_rknn()`	加载设备侧要运行的 RKNN 模型
Runtime 初始化	`init_runtime()`	初始化运行时环境
模型执行	`inference()`	对输入数据执行推理并返回结果
资源释放	`release()`	使用完成后释放 RKNNLite 对象资源

除推理接口外，RKNN Toolkit Lite2 还提供版本和平台查询接口：

```
load_rknn()
    ↓
init_runtime()
    ↓
get_sdk_version()
```

`get_sdk_version()` 只有在 Runtime 已成功初始化后才能调用；Runtime 未初始化时接口返回 `None`。由于 `init_runtime()` 本身要求模型已加载，因此正常流程仍是先 `load_rknn()`，再 `init_runtime()`，最后调用 `get_sdk_version()`。

`list_support_target_platform()` 用于查询给定 RKNN 模型可运行的芯片平台。该接口通过 `rknn_model` 参数接收 RKNN 模型路径；如果不指定模型路径，则按类别打印 RKNN Toolkit Lite2 当前支持的芯片平台。

2.3 输入数据、数据布局和输出

`inference()` 的当前接口签名为：

```
inference(
    inputs,
    data_type=None,
    data_format=None,
    inputs_pass_through=None,
    get_frame_id=False,
    reuse_outputs=False
)
```

其中 `inputs` 为输入数据列表，其他参数用于补充输入类型、数据布局、Pass Through 配置、异步模式下的 Frame ID 获取行为以及输出内存复用配置。

2.3.1 `inputs`

`inputs` 用于传入模型输入数据，类型为 `list`，列表成员为 `ndarray`。典型调用方式如下：

```
outputs = rknn_lite.inference(inputs=[img])
```

多输入模型同样通过列表传入：

```
outputs = rknn_lite.inference(
    inputs=[input0, input1]
)
```

应用应在调用推理接口前完成各输入所需的预处理。

2.3.2 `data_type` 和 `data_format`

`data_type` 用于指定输入数据类型。当前接口说明中列出的类型包括：`int8`、`uint8`、`int16`、`float16` 和 `float32`；默认参数值为 `None`。

`data_format` 用于指定输入数据布局，当前支持 `"nhwc"` 和 `"nchw"`；默认参数值为 `None`。该参数会原样传递给 Runtime 的输入设置逻辑，应用应保证其取值与实际输入 Buffer 的排列方式一致。

2.3.3 `inputs_pass_through`

`inputs_pass_through` 用于为各输入设置 Pass Through 标志。接口说明中约定使用 `0` / `1` 表示关闭 / 开启，并以列表形式对应各输入；默认值为 `None`。

2.3.4 `get_frame_id`

`get_frame_id` 默认为 `False`。为 `False` 时，接口直接返回输出 `ndarray` 列表；为 `True` 时，接口会在输出列表末尾追加 Runtime 返回的 Frame ID，主要用于异步模式下需要关联输入/输出帧的场景。

2.3.5 `reuse_outputs`

`reuse_outputs` 用于控制输出数组的内存复用方式，默认值为 `False`，该参数为可选优化项，不会改变默认行为。

为 `False`（默认）时，`inference()` 返回的输出 `ndarray` 各自拥有独立内存，跨多次调用保持有效，符合历史 API 契约，应用可安全持有返回结果。

为 `True` 时，接口直接返回内部可复用缓冲（staging buffer）的视图，每次调用不再产生大块内存分配，适用于对内存分配敏感、且输出在下次推理前已被消费完毕的长时推理循环场景。**注意**：此时返回的数组在下次调用 `inference()` 或 `get_outputs()` 时会被覆盖，如需保留结果，应在下次调用前拷贝（如 `np.array(output)`）。

2.3.6 推理输出与失败返回

正常情况下，`inference()` 返回输出 `ndarray` 列表：

```
输入 ndarray 列表
        ↓
    inference()
        ↓
输出 ndarray 列表
        ↓
    业务后处理
```

如果 Runtime 尚未初始化，或设置输入、执行模型、获取输出任一步骤发生异常，接口返回 `None`。因此业务代码建议先检查返回值，再进入后处理逻辑。

2.4 NPU Core 配置

`init_runtime()` 通过 `core_mask` 参数配置 NPU Core 工作模式，默认值为 `RKNNLite.NPU_CORE_AUTO`。

`RKNNLite` 当前定义的 Core Mask 常量如下：

core_mask	值	说明
`RKNNLite.NPU_CORE_AUTO`	`0`	自动模式，默认值。
`RKNNLite.NPU_CORE_0`	`1`	Core 0 模式。
`RKNNLite.NPU_CORE_1`	`2`	Core 1 模式。
`RKNNLite.NPU_CORE_2`	`4`	Core 2 模式。
`RKNNLite.NPU_CORE_0_1`	`3`	Core 0/1 组合模式。
`RKNNLite.NPU_CORE_0_1_2`	`7`	Core 0/1/2 组合模式；当前接口说明标注为仅 RK3588 支持。
`RKNNLite.NPU_CORE_ALL`	`0xffff`	所有 NPU Core。

源码在设置 Core Mask 的位置标注该参数只对 RK3576 / RK3588 有效。不同芯片实际支持的 Core 组合可能不同，应以对应 Runtime 的校验结果为准；如果不需要手动指定 Core，使用默认的 `RKNNLite.NPU_CORE_AUTO`。

典型初始化方式如下：

```
ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

2.5 查询接口和资源释放

RKNN Toolkit Lite2 除模型推理外，还提供设备枚举、SDK 版本查询和模型支持平台查询接口。不同接口的调用条件不同。

SDK 版本查询

`get_sdk_version()` 用于获取以下信息：

- Runtime 版本；
- 驱动版本；
- RKNN 模型版本信息。

其调用关系为：

```
load_rknn()
    ↓
init_runtime()
    ↓
get_sdk_version()
```

示例：

```
sdk_version = rknn_lite.get_sdk_version()
```

使用该接口前必须先成功完成 Runtime 初始化；否则接口返回 `None`。

模型支持平台查询

`list_support_target_platform()` 用于查询给定 RKNN 模型可运行的芯片平台：

```
rknn_lite.list_support_target_platform(
    rknn_model='mobilenet_v1.rknn'
)
```

如果 `rknn_model=None`，接口将参数直接交给平台查询函数，由其列出当前支持的平台；如果显式指定的模型路径不存在（包括空字符串路径），接口返回 `None`。

资源释放

RKNNLite 对象使用完成后，应调用：

```
rknn_lite.release()
```

从完整部署流程看，资源释放位于模型推理和相关查询操作之后：

```
RKNNLite()
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
inference() / get_sdk_version()
    ↓
release()
```

`release()` 是 RKNNLite 对象生命周期的结束步骤。完成该对象上的模型运行操作后，应及时释放相关资源。

3. RKNNLite 类

RKNNLite 是 RKNN Toolkit Lite2 的主要 Python 类，提供对象初始化、RKNN 模型加载、设备枚举、Runtime 初始化、模型推理、SDK 版本查询、模型支持平台查询和资源释放等功能。

3.1 对象初始化和释放

init

API	`RKNNLite(...)`
功能	初始化 RKNNLite 对象。可通过参数控制详细日志的屏幕输出和文件输出。
参数	`verbose`：指定是否在屏幕打印详细日志信息。
	`verbose_file`：指定日志文件路径；当设置该参数且 `verbose=True` 时，日志同时写入指定文件。
返回值	`RKNNLite` 对象。

示例：

```
# 将详细日志输出到屏幕，并写入 inference.log
rknn_lite = RKNNLite(
    verbose=True,
    verbose_file='./inference.log'
)

# 只在屏幕打印详细日志
rknn_lite = RKNNLite(verbose=True)
```

release

API	`release`
功能	使用完成后释放 RKNNLite 对象占用的资源。
参数	无
返回值	`None`。

示例：

```
rknn_lite.release()
```

使用说明：

- 在 RKNNLite 对象使用结束后调用 `release()`。
- 应保证模型推理和相关查询操作完成后再释放对象。

3.2 模型加载与 Runtime 初始化

load_rknn

API	`load_rknn`
功能	加载 RKNN 模型。
参数	`path`：RKNN 模型文件路径。
	`key_path`：（可选）加密模型的密钥信封文件路径。加载 RKCE 加密模型时必须传入，明文模型可省略。
返回值	成功：`0`；失败：`-1`。

示例（明文模型）：

```
# 从当前目录加载 resnet_18.rknn 模型
ret = rknn_lite.load_rknn('./resnet_18.rknn')
```

示例（加密模型）：

```
# 加载加密模型及其密钥信封
ret = rknn_lite.load_rknn(
    './encrypted_model.rknn',
    key_path='./encrypted_model_key.bin'
)
```

使用说明：

- `load_rknn()` 应在 `init_runtime()` 前调用。
- 模型文件应为已通过 RKNN Toolkit2 转换得到的 RKNN 模型。

加密模型说明：

RKNN Toolkit Lite2 支持加载加密后的 RKNN 模型。加密模型采用混合加密方案（AES-256-CTR + RSA-OAEP）：模型文件头部写入 RKCE 标识，模型主体使用 AES-CTR 加密；AES 密钥通过 RSA-OAEP 封装为密钥信封（RKKE 格式），与加密模型一起部署到设备端。

加载加密模型时，通过 `key_path` 参数传入密钥信封文件，运行时在 `rknn_init()` 内部自动检测 RKCE 标识并透明完成解密。解密完成后，后续推理流程与明文模型完全一致。

加密模型在 PC/服务器端使用 RKNN Toolkit2 提供的加密工具生成（需要 RK 公钥 `rk_public_key.pem`），加密后得到两个配套文件：

文件	说明
`xxx.rknn`	加密后的 RKNN 模型（RKCE 标识 + AES-CTR 加密主体）
`xxx_key.bin`	RSA-OAEP 加密的密钥信封（与加密模型配套部署）

加密模型支持的平台包括 RK3562、RK3566系列、RK3568系列、RK3576系列、RK3588系列、RV1126B、RV1103B系列、RV1106B系列。

注意事项：

- 加密模型与密钥信封必须配套使用，两者缺一不可。
- 密钥信封包含模型解密所需的密钥材料，应妥善保管，避免泄露。
- 加密方案及加密工具的更多细节，请参考《RKNN2_SDK_开发指南》中的“模型加密”相关章节。

init_runtime

API	`init_runtime`
功能	初始化运行时环境。模型推理前必须调用该接口。
参数	`target`：目标平台，默认值为 `None`。在 aarch64 设备端使用本机 NPU 时通常保持为 `None`。
	`device_id`：设备 ID，默认值为 `None`。仅在当前运行环境明确支持设备选择时使用。
	`async_mode`：是否启用异步模式，默认值为 `False`。
	`core_mask`：NPU Core 工作模式，默认值为 `RKNNLite.NPU_CORE_AUTO`。
返回值	成功返回 `0`；前置条件不满足或 Runtime 初始化失败时返回 `-1`。

`core_mask` 可使用 `RKNNLite` 类中定义的以下常量：

`core_mask`	值	说明
`RKNNLite.NPU_CORE_AUTO`	`0`	自动模式，默认值。
`RKNNLite.NPU_CORE_0`	`1`	Core 0 模式。
`RKNNLite.NPU_CORE_1`	`2`	Core 1 模式。
`RKNNLite.NPU_CORE_2`	`4`	Core 2 模式。
`RKNNLite.NPU_CORE_0_1`	`3`	Core 0/1 组合模式。
`RKNNLite.NPU_CORE_0_1_2`	`7`	Core 0/1/2 组合模式；当前接口说明标注为仅 RK3588 支持。
`RKNNLite.NPU_CORE_ALL`	`0xffff`	所有 NPU Core。

源码在 Core Mask 设置逻辑处标注 `core_mask` 只对 RK3576 / RK3588 有效。

示例：

```
ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

使用说明：

- 必须先调用 `load_rknn()` 加载模型；未加载模型时接口直接返回 `-1`。
- 如果对象上已经存在 Runtime，重新初始化前会先释放旧 Runtime。
- Runtime 创建、模型图构建、Core Mask 设置或版本检查过程中发生异常时，接口返回 `-1`。
- `init_runtime()` 成功后才能调用 `inference()`；`get_sdk_version()` 同样要求 Runtime 已初始化。

3.3 模型推理

inference

API	`inference`
功能	对指定输入执行模型推理，并返回输出结果。
参数	`inputs`：输入数据，类型为 `list`，成员为 `ndarray`。
	`data_type`：输入数据类型，默认值为 `None`；接口说明列出的类型包括 `int8`、`uint8`、`int16`、`float16`、`float32`。
	`data_format`：输入数据布局，默认值为 `None`；当前支持 `"nhwc"` 和 `"nchw"`。
	`inputs_pass_through`：各输入的 Pass Through 标志列表，元素使用 `0` / `1` 表示关闭 / 开启，默认值为 `None`。
	`get_frame_id`：是否获取 Frame ID，默认值为 `False`；主要用于异步模式。
	`reuse_outputs`：是否复用内部输出缓冲，默认值为 `False`；为 `True` 时返回内部复用缓冲的视图，适用于长时推理循环场景，但下次推理会覆盖返回结果。
返回值	正常返回输出 `ndarray` 列表；`get_frame_id=True` 时在列表末尾追加 Frame ID。Runtime 未初始化或推理过程中发生异常时返回 `None`。

示例：

```
outputs = rknn_lite.inference(inputs=[img])
if outputs is None:
    print('Inference failed')
else:
    show_top5(outputs)
```

长时推理循环中如需降低内存分配，可开启输出复用：

```
for frame in frames:
    outputs = rknn_lite.inference(
        inputs=[frame],
        reuse_outputs=True
    )
    show_top5(outputs)          # 用完即弃，结果在下次推理前消费完毕
```

参考输出：

```
-----TOP 5-----
[812]: 0.999442994594574
[404]: 0.0004096269840374589
[657]: 3.284541890025139e-05
[833]: 2.6112385967280716e-05
[895]: 1.8509887013351545e-05
```

使用说明：

- 调用前必须先成功执行 `init_runtime()`；否则返回 `None`。
- `inputs`、`data_type`、`data_format` 和 `inputs_pass_through` 会传递给 Runtime 的输入设置逻辑。
- 设置输入、执行模型或获取输出过程中发生异常时，接口返回 `None`。
- 当 `get_frame_id=False` 时返回普通输出列表；当 `get_frame_id=True` 时，在输出列表末尾追加运行返回的 Frame ID。
- `reuse_outputs` 默认为 `False`，返回数组拥有独立内存，跨调用有效；仅在明确需要降低内存分配时开启 `reuse_outputs=True`，此时返回的是内部复用缓冲视图，会被下次推理覆盖，需保留结果时应先拷贝。
- 推理成功后，再根据模型类型执行相应后处理。

3.4 设备、版本和平台查询

list_devices

API	**`list_devices`**
功能	枚举当前可通过 ADB 和 NTB 访问的设备，并打印设备列表。
参数	无
返回值	`(adb_devices, ntb_devices)`：两个 `list`。同时出现在 NTB 列表中的设备会从 ADB 返回列表中移除。

示例：

```
adb_devices, ntb_devices = rknn_lite.list_devices()
```

get_sdk_version

API	**`get_sdk_version`**
功能	获取 SDK 版本信息。
参数	无
返回值	正常返回 `sdk_version` 字符串；Runtime 未初始化或查询失败时返回 `None`。

调用该接口前必须先成功完成 Runtime 初始化。`init_runtime()` 又要求模型已加载，因此标准顺序为 `load_rknn()` → `init_runtime()` → `get_sdk_version()`。

示例：

```
sdk_version = rknn_lite.get_sdk_version()
if sdk_version is None:
    print('Get SDK version failed')
```

参考返回信息：

```
I RKNN: [10:47:23.097] RKNN Runtime Information:
librknnrt version: 1.2.0b1
(a47985372@2021-12-06T10:20:14)

I RKNN: [10:47:23.097] RKNN Driver Information:
version: 0.6.2

I RKNN: [10:47:23.098] RKNN Model Information:
version: 1,
toolkit version: 1.1.2
(compiler version: 1.1.2b13
(1e5726f57@2021-11-29T14:13:55)),
target: RKNPU v2,
target platform: rk3588,
framework name: PyTorch,
framework layout: NCHW
```

list_support_target_platform

API	`list_support_target_platform`
功能	查询给定 RKNN 模型可运行的芯片平台，或列出当前支持的平台。
参数	`rknn_model`：RKNN 模型路径，默认值为 `None`。为 `None` 时直接调用平台查询函数列出当前支持的平台。
返回值	正常返回 `support_target`，类型为 `dict`；显式指定的模型路径不存在时返回 `None`。

示例：

```python
support_target = rknn_lite.list_support_target_platform(
    rknn_model='mobilenet_v1.rknn'
)
if support_target is None:
    print('Query target platform failed')
```

参考结果：

```
************************************************
Target platforms filled in RKNN model: ['RK3588']
Target platforms supported by this RKNN model: ['RK3588']
************************************************
```

使用说明：

- `rknn_model=None` 时不会触发文件存在性检查，而是直接调用底层平台查询函数。
- 当显式指定的模型路径不存在时，接口记录错误并返回 `None`。

4. 异常与错误处理

RKNN Toolkit Lite2 Python 接口主要通过状态码、`None` 返回值和运行日志反映执行状态。状态型接口应显式检查返回值，推理和查询类接口应在使用结果前检查是否为 `None`。

4.1 模型加载和 Runtime 初始化错误

`load_rknn()` 和 `init_runtime()` 的返回值约定为：

- 成功：`0`
- 失败：`-1`

推荐在每一步初始化后立即检查：

```
ret = rknn_lite.load_rknn('./resnet_18.rknn')
if ret != 0:
    print('Load RKNN model failed')
    exit(ret)

ret = rknn_lite.init_runtime(
    core_mask=RKNNLite.NPU_CORE_AUTO
)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

只有模型加载和 Runtime 初始化成功后，才应继续调用 `inference()` 或 `get_sdk_version()`。

4.2 推理输入与返回值检查

调用 `inference()` 前应重点确认：

- 已成功调用 `init_runtime()`；
- `inputs` 为列表，成员为模型需要的 `ndarray`；
- 如使用 `data_type`、`data_format` 或 `inputs_pass_through`，其取值与实际输入保持一致；
- 异步模式下如需关联帧信息，可设置 `get_frame_id=True`；
- 如开启 `reuse_outputs=True`，返回数组为内部复用缓冲视图，使用后不跨调用持有，需要保留结果时先拷贝。

`inference()` 在以下情况下返回 `None`：Runtime 未初始化、设置输入失败、模型运行失败或获取输出失败。因此建议统一进行空值检查：

```
outputs = rknn_lite.inference(inputs=[input_data])
if outputs is None:
    print('Inference failed')
    exit(-1)
```

4.3 NPU Core 配置检查

源码在 Core Mask 设置逻辑处标注 `core_mask` 只对 RK3576 / RK3588 有效；同时，`RKNNLite.NPU_CORE_0_1_2` 在接口说明中标注为仅 RK3588 支持。

如果不需要手动指定 Core，建议使用默认的 `RKNNLite.NPU_CORE_AUTO`。对于特定芯片和 Core 组合，应以 Runtime 的实际校验结果为准；Core Mask 设置失败时，`init_runtime()` 返回 `-1`。

4.4 日志输出

初始化 `RKNNLite` 时可使用 `verbose` 和 `verbose_file` 输出详细日志：

```
rknn_lite = RKNNLite(
    verbose=True,
    verbose_file='./inference.log'
)
```

当 `verbose=True` 时，详细日志会打印到屏幕；如果同时设置 `verbose_file`，日志还会写入指定文件。

出现模型加载、Runtime 初始化或推理问题时，可结合接口返回值和详细日志定位问题。

5. 附录

5.1 参考文档

《RKNN2_SDK_开发指南》：RKNN2 软件栈、模型转换与量化、部署流程及故障排查。

《RKNN2_Toolkit_Python_API_参考》：RKNN-Toolkit2 Python API 方法、参数及返回值说明。

RKNN Toolkit2 相关文档：

https://github.com/airockchip/rknn-toolkit2/tree/master/doc

RKNN Toolkit2 项目：

https://github.com/airockchip/rknn-toolkit2

5.2 问题反馈渠道

可通过以下渠道向 Rockchip NPU 团队反馈问题：

- RKNN QQ 交流群：1077888690
- GitHub Issue：https://github.com/airockchip/rknn-toolkit2/issues
- Rockchip Redmine：https://redmine.rock-chips.com/

注：Redmine 账号需要通过销售或业务人员开通。如果使用第三方开发板，请先向对应开发板厂商反馈问题。